

Print Debugging

Print debugging is the practice of placing temporary `print()` calls in a program to display the values of variables and then rerunning the program. The process often follows these steps:

1. Notice a bug in your program.
2. Add `print()` calls for some variables to find out what they contain.
3. Rerun the program.
4. Add some more `print()` calls because the earlier ones didn't show enough information.
5. Rerun the program.
6. Repeat the previous two steps a few more times before finally figuring out the bug.
7. Rerun the program.
8. Realize you forgot to remove some of the `print()` calls and remove them.

Print debugging is deceptively quick and simple. But it often requires multiple iterations of rerunning the program before you display the information you need to fix your bug. The solution is to use a debugger or set up logfiles for your program. By using a debugger, you can run your programs one line of code at a time and inspect any variable. Using a debugger might seem slower than simply inserting a `print()` call, but it saves you time in the long run.

Logfiles can record large amounts of information from your program so you can compare one run of it to previous runs. In Python, the built-in `logging` module provides the functionality you need to easily create logfiles by using just three lines of code:

```
import logging
logging.basicConfig(filename='log_filename.txt', level=logging.DEBUG, format='%(asctime)s - %(levelname)s - %(message)s')
logging.debug('This is a log message.')
```
